



Rapport de projet : OCR Word Search Solver

GREINER Matthieu **TEIXEIRA** Julien **CHAMBON** Pierre-Louis **BARROS** Dylan



FIGURE 1 – Image du groupe SegFaultless

Table des matières

1	Introduction	2
2	Le groupe	3
2.1	SegFaultless	3
2.2	Les membres de l'équipe	3
3	Le projet	4
3.1	Informations globales sur le projet	4
3.2	Procédures d'exécution du projet	5
4	La répartition des tâches	5
4.1	Les caractéristiques principales du projet	5
4.2	Pour notre première soutenance	6
4.3	Répartition des tâches	7
5	Aspects techniques	7
5.1	Spécifié par le cahier des charges	7
5.1.1	Environnement de développement	7
5.1.2	Options de compilation et normes de code	7
5.1.3	Gestion du dépôt git	7
5.1.4	Ressources visuelles	7
5.1.5	Exclusions du dépôt	8
5.2	Mis en place par nous mêmes	8
5.2.1	Les bibliothèques et les logiciels utilisés	8
5.2.2	Le chargement d'une image et suppression des couleurs	9
5.2.3	La rotation manuelle de l'image	9
5.2.4	La détection de la position	10
5.2.5	Le découpage de l'image	10
5.2.6	L'implémentation de l'algorithme de résolution d'une grille de mots cachés	10
5.2.7	La preuve de concept d'un réseau de neurones	11
5.2.8	Les tâches restantes	11
6	États d'avancement du projet	12
6.1	Le chargement d'une image et suppression des couleurs	12
6.2	La rotation manuelle de l'image	12
6.3	La détection de la position	12
6.4	Le découpage de l'image	13
6.5	La preuve de concept d'un réseau de neurones	13
6.6	Tableau d'avancement des tâches	13
7	Conclusion	13
A	Annexes	14

1 Introduction

Dans un monde où l'automatisation est de plus en plus présente, la reconnaissance optique de caractères (OCR) s'impose comme une technologie incontournable, permettant de transformer des images de textes en données numériques exploitables. Que ce soit pour numériser des documents, extraire des informations ou faciliter des recherches dans des fichiers scannés, l'OCR est devenu un outil essentiel dans de nombreux secteurs, de l'administration aux applications mobiles en passant par les outils de recherche en entreprise. Notre équipe, SegFaultless, a entrepris de concevoir un système OCR dans le cadre de notre projet universitaire, avec pour objectif de résoudre un problème spécifique : l'identification et la résolution de grilles de mots cachés à partir d'images. Ce projet représente un défi passionnant, alliant manipulation d'images, algorithmie avancée et intelligence artificielle.

Le processus que nous avons mis en place pour atteindre cet objectif repose sur plusieurs étapes clés. La première consiste à charger l'image et supprimer les couleurs superflues pour simplifier le traitement. Nous avons déjà développé une fonction de chargement d'images aux formats courants tels que PNG et JPEG en utilisant la bibliothèque SDL. Cependant, pour une meilleure efficacité de traitement, il est également nécessaire de convertir ces images en niveaux de gris, ce qui facilite les opérations de détection de contours et de reconnaissance de motifs.

Une autre fonctionnalité essentielle déjà en place est la rotation manuelle de l'image. Notre système permet de pivoter l'image par intervalles de 90 degrés, selon les indications de l'utilisateur. Cette fonctionnalité est particulièrement utile pour aligner correctement la grille de mots cachés et garantir un traitement précis lors des étapes suivantes.

La phase suivante implique le découpage de l'image en sous-parties représentant chaque lettre. Cette tâche, bien qu'encore en cours de développement, est cruciale pour alimenter notre système OCR. En découplant chaque lettre en une image individuelle, nous faciliterons l'entrée de données pour l'algorithme de reconnaissance de caractères, améliorant ainsi la précision de l'identification des lettres dans la grille.

Notre équipe a également consacré des efforts significatifs au développement de l'algorithme de résolution de la grille de mots cachés. Ce module, accessible via la ligne de commande, utilise une approche de recherche de mots pour détecter les termes présents dans la grille. Nous avons commencé avec une solution simple, que nous prévoyons d'optimiser pour gagner en efficacité et en rapidité.

Enfin, en parallèle de notre système OCR, nous avons développé une preuve de concept d'un réseau de neurones. Bien que modeste, ce réseau a pour but de démontrer notre capacité à intégrer des techniques d'intelligence artificielle dans notre projet. Cette approche pourrait, à terme, nous permettre d'améliorer la précision de notre système OCR en utilisant l'apprentissage automatique pour affiner la reconnaissance des lettres et des mots dans la grille.

En conclusion, le projet OCR de SegFaultless est une entreprise ambitieuse qui requiert une maîtrise des techniques de traitement d'image et d'algorithmes de recherche, ainsi qu'une compréhension des réseaux de neurones. Chaque étape, qu'il s'agisse de la manipulation d'images ou de l'implémentation d'algorithmes, nous rapproche de notre objectif final : concevoir un outil capable de résoudre automatiquement des grilles de mots cachés à partir d'images. Cette expérience enrichissante nous permet d'explorer les multiples facettes de l'OCR tout en contribuant au développement de compétences précieuses pour notre avenir professionnel.

2 Le groupe

2.1 SegFaultless

Le choix du nom SegFaultless pour notre groupe est à la fois un clin d'œil technique et un engagement envers la qualité de notre travail. Inspiré du terme "Segmentation Fault" (ou "SegFault"), une erreur courante en programmation indiquant un accès mémoire non autorisé, notre nom reflète notre volonté d'éviter les erreurs critiques et de créer un projet "faultless" — sans défaut. Ce jeu de mots incarne notre ambition de dépasser les obstacles techniques tout en produisant un travail rigoureux, stable et professionnel.

2.2 Les membres de l'équipe

Le groupe **SegFaultless** est composé de quatre membres aux compétences complémentaires, chacun jouant un rôle clé dans le projet OCR Word Search Solver. Voici les membres de l'équipe :

— **Julien :**

Je m'appelle Julien et je suis passionné par le développement logiciel et l'intelligence artificielle. Ces domaines m'inspirent énormément car ils nous permettent de résoudre des problèmes complexes de manière créative tout en ouvrant la voie à des avancées technologiques qui ont un impact sur la vie quotidienne. Dans ce projet, les aspects techniques qui m'ont le plus attiré étaient la programmation et l'optimisation des algorithmes. Ces domaines me permettront d'améliorer mes compétences en rédaction de code efficace, en construction de projets complexes et en résolution de problèmes en temps réel. Travailler en équipe est pour moi une grande source de motivation. Non seulement cela m'a donné une nouvelle perspective sur les projets, mais cela m'a également offert un environnement de travail dynamique où chacun pouvait apporter ses forces et ses idées. Nous avons décidé de travailler ensemble car nous partageons la même ambition.

— **Pierre Louis :**

J'ai découvert l'informatique à l'âge de 9 ans grâce au jeu vidéo Minecraft. J'ai très vite eu envie de comprendre ce qu'il se passait derrière pour qu'un jeu comme Minecraft puisse fonctionner. C'est à l'âge de 12 ans que j'ai commencé la programmation dans le but de développer un serveur minecraft avec différents mini-jeux. Depuis je n'ai jamais quitté la programmation. Durant mon parcours, j'ai hésité entre médecine et informatique, aujourd'hui moi but et de combiner ces deux secteurs en faisant de l'informatique en lien avec le corps de la Santé, je suis particulièrement intéressé par les réseaux de neurone d'analyse d'image qui adapté à la médecine peuvent permettre par exemple de détecter des cancers à des stades très précoces. Ce projet d'OCR est donc extrêmement important pour moi et je suis prêt à me donner à 200% pour mener à bien ce projet.

— **Dylan :**

Depuis que j'ai découvert le développement de jeux, je suis devenu passionné par la technologie et ses applications innovantes, notamment dans des domaines comme l'intelligence artificielle et l'analyse d'images. Cet intérêt m'a conduit à explorer des sujets comme les réseaux de neurones, dont les capacités d'analyse sont particulièrement prometteuses, notamment en médecine. Par exemple, je suis fasciné par la manière dont les réseaux neuronaux peuvent contribuer à la détection précoce de maladies graves, comme le cancer, où une détection rapide peut faire une réelle différence pour les patients.

Bien que je n'aie pas encore eu l'occasion de travailler sur un projet de reconnaissance optique de caractères (OCR), j'ai hâte de m'y mettre. Pour moi, ce sera une occasion en or d'approfondir mes connaissances en traitement d'image et en IA. Je suis prêt à m'investir pleinement et à relever tous les défis, afin de réussir ce projet et de développer des compétences essentielles dans ces domaines.

Bien que je n'aie pas encore eu l'occasion de travailler sur un projet de reconnaissance optique de caractères (OCR), j'ai hâte de m'y mettre. Pour moi, ce sera une occasion en or d'approfondir mes connaissances en traitement d'image et en IA. Je suis prêt à m'investir pleinement et à relever tous les défis, afin de réussir ce projet et de développer des compétences essentielles dans ces domaines.

— **Matthieu :**

Passionné par les aspects fondamentaux de la programmation en C, j'ai trouvé dans l'OCR une application concrète qui met à l'épreuve les compétences acquises lors de ma formation à l'EPITA. Ce projet me permet de manipuler des concepts clés en traitement d'image et en gestion mémoire, des domaines techniques que j'ai particulièrement explorés et qui m'enthousiasment. Ces compétences, appliquées ici dans le cadre d'un projet de groupe, renforcent mes bases pour des applications en vision par ordinateur et en reconnaissance de motifs, des domaines qui me motivent particulièrement.

Sur le plan personnel, l'OCR m'offre une opportunité unique de collaborer avec une équipe que j'apprécie pour son professionnalisme et son ambition. Nous avons décidé de travailler ensemble car chacun apporte des compétences spécifiques et une perspective complémentaire, ce qui rend le travail de groupe à la fois stimulant et enrichissant. Cette expérience nous permet non seulement de progresser techniquement, mais aussi de développer des compétences humaines essentielles, comme la communication et la gestion collaborative d'un projet complexe.

Chaque membre apporte son expertise et sa motivation pour mener à bien ce projet ambitieux, garantissant que toutes les étapes, de l'analyse des images à la résolution de la grille de mots, soient exécutées avec rigueur et efficacité.

3 Le projet

3.1 Informations globales sur le projet

Le projet de reconnaissance optique de caractères (OCR) a pour objectif de développer un système capable de détecter et de lire des lettres et des mots à partir d'images de grilles de mots cachés. Ce projet est réalisé en langage C et utilise la bibliothèque SDL pour le traitement et l'affichage des images.

Les principales fonctionnalités du projet incluent :

- Chargement d'images dans des formats courants (PNG, JPEG).
- Prétraitement des images pour améliorer la lisibilité (conversion en niveaux de gris, suppression des couleurs).
- Détection de la grille et des lettres.
- Découpage de l'image pour extraire chaque lettre en tant qu'image individuelle.
- Implémentation d'un algorithme de résolution de grilles de mots cachés.
- Démonstration d'un mini réseau de neurones pour une tâche logique simple.

Le projet vise également à respecter des contraintes de performance et de compatibilité, en s'assurant qu'il fonctionne sur les systèmes NixOS et MacOS. Les résultats seront présentés sous forme d'une interface utilisateur simple permettant de visualiser les grilles et les mots détectés.

3.2 Procédures d'exécution du projet

Pour exécuter le projet OCR, plusieurs étapes doivent être suivies :

1. Installation des dépendances :

- Assurez-vous d'avoir la bibliothèque SDL installée sur votre système. Vous pouvez l'installer via le gestionnaire de paquets Nix ou MacOS.

2. Préparation des images :

- Placez les images que vous souhaitez traiter dans le dossier `data/examples` du projet. Assurez-vous que les images sont dans un format compatible (PNG, JPEG).

3. Compilation du code :

- Utilisez le Makefile fourni pour compiler le projet. Exécutez la commande suivante dans le terminal :
 - `make ocrgr` ou `make ocrb` pour compiler uniquement le chargement de l'image avec soit le traitement avec nuances de gris soit le traitement noir et blanc (par dessus les nuances de gris).
 - `make solver` pour compiler le solver manuel.
 - `make neural` pour compiler la fonction logique.
 - Et aussi `make all` si l'on veut tout compiler à la fois.

5. Exécution de l'application :

- Une fois la compilation réussie, exécutez le programme avec la commande suivante, en spécifiant le fichier image à traiter : `./bin/ data/examples/<image_file>`
- Pour le solver ça sera, `./bin grid mot voulu`
- Pour le réseau de neurones, `./bin neural_test A B`, A et B étant des variables prenant leurs valeurs dans 0 et 1.

6. Affichage des résultats :

- Après le traitement, dans le terminal seront affichés des informations comme la rotation automatique de la grille mais aussi les informations sur la fonction logique et le réseau de neurones et pour le chargement de l'image une fenêtre apparaîtra pour montrer l'image avec les différents niveaux de gris.

Ces étapes assurent que le projet est exécuté de manière fluide, permettant aux utilisateurs de tirer pleinement parti de ses fonctionnalités de reconnaissance de caractères.

4 La répartition des tâches

4.1 Les caractéristiques principales du projet

Le projet OCR se concentre sur la détection de mots dans des images de grilles de mots cachés. Les principales caractéristiques du projet incluent :

- Chargement d'images dans des formats variés (PNG, JPEG) pour un traitement ultérieur.

- Prétraitement des images, y compris la conversion en niveaux de gris pour simplifier l'analyse.
- Détection et extraction de la grille de mots, des lettres et des mots de la liste à l'aide d'algorithmes adaptés.
- Résolution des grilles de mots cachés par un module dédié, utilisant des méthodes de recherche optimisées.
- Implémentation d'une preuve de concept pour un mini réseau de neurones, illustrant une approche d'apprentissage simple.

4.2 Pour notre première soutenance

Pour notre première soutenance, nous avons réalisé plusieurs avancées significatives :

- **Chargement des images :** Nous avons implémenté une fonction pour charger des images (formats PNG et JPEG) en utilisant la bibliothèque SDL. Cette étape est essentielle pour préparer le traitement des images.
- **Rotation manuelle de l'image :** Nous avons développé une fonction qui permet de faire pivoter l'image manuellement en capturant l'entrée de l'utilisateur pour ajuster l'angle. Cela facilite le traitement ultérieur des images qui peuvent être orientées incorrectement.
- **Détection de contours :** Nous avons commencé à travailler sur la détection de contours, bien que cette fonctionnalité reste à finaliser. L'objectif est de distinguer la grille et les lettres pour une meilleure précision dans la reconnaissance.
- **Découpage de l'image :** Nous avons prévu de découper chaque lettre détectée et de l'enregistrer en tant qu'image individuelle, ce qui facilitera l'entrée pour l'étape OCR.
- **Module de résolution de mots :** Nous avons réussi à créer un module de ligne de commande pour parcourir la grille et détecter les mots de la liste. Ce module a été conçu avec un algorithme simple de recherche de mots, qui sera optimisé dans les prochaines étapes.
- **Preuve de concept du réseau de neurones :** Nous avons également développé une preuve de concept pour un mini réseau de neurones, montrant comment il peut apprendre à réaliser des fonctions logiques simples.

4.3 Répartition des tâches

Voici le tableau présentant la répartition des tâches dans notre groupe de projet. Il faut aussi noter que ces tâches ne sont pas vraiment respectives sachant que tout le monde s'entraidait sur tous les sujets.

	Matthieu	Dylan	Julien	Pierre-Louis
Chargement de l'image	X			
Rotation manuelle de l'image	X			X
Suppression des couleurs	X		X	X
Détection de la grille et des éléments		X	X	
Segmentation de l'image		X	X	X
Module de résolution des mots d'une grille	X			
Développement de la fonction logique du réseau de neurones	X			X
Compilation Makefiles	X		X	X
Rapport de soutenance	X	X	X	X

TABLE 1 – Tableau détaillé présentant la répartition des tâches au sein du groupe.

5 Aspects techniques

5.1 Spécifié par le cahier des charges

5.1.1 Environnement de développement

Pour l'environnement de développement nous avons particulièrement codé sur WSL qui est une machine virtuelle simulant une distro Linux ou particulièrement Ubuntu dans notre cas et nous avons utilisé Visual Studio Code pour gérer les Makefiles et les fichiers C. De plus nous avons aussi testés entièrement notre code sur les machines des salles machines en mode prépa pour s'assurer du bon fonctionnement intégral du projet sous NixOS.

5.1.2 Options de compilation et normes de code

Les options de compilation indiquées par le cahier de charges sont au moins de compiler avec les flags - Wall -Wextra. De plus le code est limité à 80 caractères par ligne pour faciliter la lecture de celui-ci.

5.1.3 Gestion du dépôt git

Le dépôt Git doit contenir absolument un Makefile final avec les commandes `all` et `clean` et le code nécessaire à la compilation du projet. Le dépôt devra contenir aussi les 2 rapports de soutenances à la racine du dépôt et étant intitulé : `rapport_n°.pdf`. De plus, il devra absolument posséder un fichier `AUTHORS` et `README`.

5.1.4 Ressources visuelles

Le dépôt GIT devra aussi contenir quelques images qui pourront servir aux démonstrations des différentes parties. Il faudra également un jeu d'images pour l'apprentissage de votre réseau de neurone.

5.1.5 Exclusions du dépôt

Pour garder le dépôt Git propre et éviter d'ajouter des fichiers inutiles, nous avons défini certaines exclusions dans un fichier `.gitignore`. Voici les principales catégories de fichiers exclus de notre dépôt :

- **Fichiers compilés et exécutables** : Les fichiers objets et exécutables générés lors de la compilation, comme les fichiers `*.o`, `*.out`, `*.exe`, ainsi que les exécutables spécifiques tels que `main_solver` et `main_neural_net`.
- **Dossiers de build** : Si des dossiers sont dédiés aux fichiers compilés, comme `build/` ou `bin/`, ils sont également exclus. Le dossier `*.dSYM/`, généré pour le débogage sur macOS, est aussi exclu.
- **Fichiers temporaires et de sauvegarde** : Les fichiers de sauvegarde ou temporaires, comme `*`, `*.swp`, `*.swo`, `*.log`, et `*.tmp`, sont ignorés pour éviter de pousser des versions inutiles de fichiers.
- **Fichiers de configuration spécifiques à l'environnement de développement** : Les configurations propres aux IDE, comme `.vscode/` et `*.code-workspace`, ainsi que les fichiers système tels que `.DS_Store` (sous macOS), sont également exclus.
- **Fichiers générés par LaTeX** : Les fichiers générés lors de la compilation LaTeX, comme `*.aux`, `*.log`, `*.out`, `*.toc`, ainsi que le fichier PDF final `*.pdf`, sont ignorés si le document est régénéré à chaque mise à jour.

5.2 Mis en place par nous mêmes

5.2.1 Les bibliothèques et les logiciels utilisés

Dans le cadre de notre projet OCR, nous avons fait appel à plusieurs bibliothèques et logiciels pour répondre aux exigences techniques et simplifier le développement :

- **SDL (Simple DirectMedia Layer)** : Nous utilisons SDL pour gérer l'affichage et le traitement des images. Cette bibliothèque graphique est essentielle pour charger les images (formats PNG et JPEG), ainsi que pour réaliser des transformations comme la rotation et le passage en niveaux de gris, tout en étant compatible avec les exigences de notre environnement de développement.
- **VS Code (Visual Studio Code)** : Pour le développement du projet, nous avons choisi VS Code comme environnement de développement intégré (IDE). Il offre des fonctionnalités avancées, telles que la gestion de versions avec Git, l'intégration de multiples extensions, et un terminal intégré, facilitant le travail collaboratif et le suivi des versions.
- **Make** : Nous utilisons des fichiers Makefile pour automatiser la compilation du code et organiser le projet en différents modules. Nous avons mis en place plusieurs Makefiles pour compiler indépendamment le module de résolution de mots (solver) et la preuve de concept du réseau de neurones, en plus du Makefile principal.
- **GCC (GNU Compiler Collection)** : Notre projet étant en C, nous avons utilisé GCC pour la compilation. Ce compilateur est compatible avec les systèmes Unix, ce qui facilite le déploiement sur NixOS et macOS.
- **Git et GitHub** : La gestion de version est effectuée avec Git, et le code est hébergé sur GitHub. Cette plateforme nous permet de suivre l'évolution du projet, de gérer les contributions de chaque membre de l'équipe et de partager le projet de manière centralisée.
- **LaTeX** : Pour la documentation et le rapport de projet, nous utilisons LaTeX. Cet outil de mise en forme permet de structurer notre rapport de manière professionnelle, tout en facilitant l'inclusion d'images, de tableaux et de formules mathématiques.

En combinant ces bibliothèques et logiciels, nous avons pu créer un environnement de développement efficace pour l'implémentation des différentes fonctionnalités du projet OCR. Cette organisation nous permet de travailler de manière modulaire et collaborative, tout en respectant les contraintes techniques imposées par le projet.

5.2.2 Le chargement d'une image et suppression des couleurs

La première étape de notre projet consiste à charger une image à partir d'un fichier afin de pouvoir la traiter. Pour cela, nous avons implémenté une fonction qui utilise la bibliothèque SDL pour ouvrir le fichier d'image et le convertir en une texture exploitable pour le traitement.

La fonction vérifie tout d'abord si l'image a été chargée correctement. Une fois cette vérification effectuée, elle convertit l'image en niveaux de gris pour faciliter les étapes de traitement ultérieures. Ensuite, l'image est transformée en texture, ce qui nous permet de l'afficher ou de l'analyser dans notre programme. Cette texture est utilisée comme point de départ pour la détection et le traitement de la grille de mots cachés.

Afin de simplifier le traitement des images, nous appliquons une conversion en niveaux de gris aux images chargées. Cette étape de suppression des couleurs consiste à transformer chaque pixel en une intensité de gris, ce qui réduit la complexité de l'image en supprimant les informations de couleur.

La fonction de prétraitement utilise la bibliothèque SDL pour parcourir les pixels de l'image d'origine et calculer leur équivalent en niveaux de gris. Pour chaque pixel, les composantes rouge, verte et bleue sont extraites. Ensuite, une valeur de luminosité est calculée en appliquant la formule de luminance, qui prend en compte la sensibilité de l'œil humain à chaque composante de couleur :

$$\text{gris} = 0.3 \times \text{rouge} + 0.59 \times \text{vert} + 0.11 \times \text{bleu}$$

Chaque pixel de l'image est ainsi remplacé par une teinte de gris correspondant à cette valeur de luminosité. Cette transformation permet de réduire les informations visuelles à un seul canal de couleur, facilitant ainsi les traitements ultérieurs de reconnaissance de caractères.

Par dessus la conversion en nuances de gris, nous avons appliqué une conversion en noir et blanc. Pour se faire, il fallait comparer chaque valeur RGB des pixels préalablement convertis en nuances de gris avec une valeur seuil qui permet de définir si on met le pixel sur blanc ou sur noir. Par exemple dans notre code nous avons utilisé la valeur 140 comme seuil, ainsi si la valeur RGB du pixel est inférieure à 140, le pixel sera défini sur la couleur noir (0) et il sera défini sur la couleur blanche (255) dans le cas contraire.

5.2.3 La rotation manuelle de l'image

La rotation manuelle de l'image permet de s'assurer que l'orientation de la grille de mots cachés est correcte avant d'appliquer les traitements de reconnaissance de caractères. Cette étape est essentielle, car un mauvais alignement peut fausser les résultats, notamment en termes de position des lettres dans la grille et de détection des mots.

Principe de fonctionnement : Pour garantir une rotation optimale, le programme demande à l'utilisateur de saisir un angle en degrés. Une fois cet angle fourni, une transformation de l'image est effectuée. Afin de conserver la qualité et la dimension des informations présentes, la surface de l'image est ajustée pour s'adapter à l'angle spécifié, sans perte ni distorsion des informations.

En permettant à l'utilisateur de spécifier l'angle de rotation, le programme offre une flexibilité qui s'adapte aux spécificités de chaque image. Cette interaction utilisateur assure une orientation précise de la grille de mots cachés, ce qui facilite par la suite la détection des mots et des lettres sans erreurs dues à un mauvais alignement.

5.2.4 La détection de la position

La détection de la position de la grille dans l'image représente une tâche complexe et cruciale pour le succès de notre projet de résolution de mots cachés. Cette étape consiste à identifier avec précision les contours et les lignes de la grille, ce qui nécessite une analyse approfondie des caractéristiques de l'image.

Jusqu'à présent, nous avons commencé à réfléchir aux différentes méthodes que nous pourrions utiliser pour réaliser cette détection. Bien que nous n'ayons pas encore finalisé notre approche, nous avons développé un exemple de code en annexe qui illustre nos premières tentatives et idées. Ce code constitue un point de départ sur lequel nous pourrions bâtir et affiner notre méthode de détection.

Il est important de noter que cette tâche implique plusieurs défis, notamment la gestion des variations d'éclairage, de couleur et de texture dans les images. Nous devons également tenir compte des différentes tailles et formes de grilles qui pourraient apparaître dans les images traitées. Ainsi, le chemin vers une détection robuste et efficace reste encore à explorer et à développer.

5.2.5 Le découpage de l'image

Le découpage de l'image est une étape essentielle de notre projet, permettant d'extraire chaque lettre de la grille afin de les traiter individuellement. Cependant, en raison des contraintes de temps, nous n'avons pas encore eu l'opportunité de finaliser cette tâche.

Nous avons commencé à réfléchir à différentes méthodes pour réaliser ce découpage. Parmi celles-ci, nous envisageons d'utiliser des techniques de détection de contours, qui pourraient nous aider à identifier précisément les lettres à partir de l'image traitée. De plus, nous devons également considérer les différentes polices et styles d'écriture qui pourraient affecter la forme et la taille des lettres.

Bien que nous n'ayons pas encore mis en œuvre de solution définitive, notre exploration préliminaire nous a permis de poser les bases d'un découpage efficace. Nous sommes conscients que cette étape nécessite une attention particulière pour garantir la précision du traitement ultérieur des lettres.

5.2.6 L'implémentation de l'algorithme de résolution d'une grille de mots cachés

L'algorithme de résolution pour la grille de mots cachés est conçu pour lire une grille de caractères à partir d'un fichier et rechercher un mot donné dans toutes les directions possibles. Ce processus est divisé en plusieurs étapes pour assurer l'efficacité de la recherche et la clarté des résultats.

— Chargement de la Grille :

La première étape consiste à charger la grille de caractères à partir d'un fichier spécifié. Le programme utilise la fonction `read_grid`, qui lit chaque ligne du fichier, convertit les caractères en majuscules et enregistre les dimensions de la grille. Cela permet d'assurer une uniformité des caractères pour la recherche des mots.

- **Recherche du Mot dans Toutes les Directions :**

Une fois la grille chargée, le programme passe à la fonction `find_word_in_grid` pour rechercher le mot cible dans toutes les directions possibles (haut, bas, gauche, droite, et les quatre diagonales). Chaque lettre est comparée à la première lettre du mot. Si une correspondance est trouvée, la fonction vérifie les lettres suivantes dans la direction choisie pour valider l'intégralité du mot.

- **Affichage des Résultats :**

Si le mot est trouvé, le programme affiche les coordonnées de début et de fin du mot dans la grille, facilitant ainsi la localisation visuelle pour l'utilisateur. Si le mot n'est pas trouvé, un message "Not Found" est affiché, signalant que le mot n'existe pas dans la grille.

5.2.7 La preuve de concept d'un réseau de neurones

La preuve de concept pour le réseau de neurones de ce projet se concentre sur la réalisation d'une opération logique spécifique, visant à prédire la sortie de la fonction $\overline{A} \cdot \overline{B} + A \cdot B$ en fonction des valeurs booléennes (0 ou 1) fournies en entrée. Le réseau est conçu avec une structure simple, où les poids et biais sont initialisés aléatoirement, puis ajustés pour améliorer la précision de cette tâche.

- **Chargement des Entrées et Initialisation du Réseau :**

Le programme commence par l'initialisation des poids et des biais avec des valeurs aléatoires, permettant au réseau de démarrer avec une certaine variabilité dans ses calculs. Ensuite, le réseau s'entraîne sur les quatre combinaisons possibles de valeurs pour A et B en utilisant la méthode de rétro propagation pour ajuster progressivement les poids et les biais.

- **Entraînement et Précision :**

Le réseau est entraîné sur chaque combinaison d'entrée (0,0 ; 0,1 ; 1,0 ; 1,1) pendant plusieurs milliers d'itérations pour affiner les paramètres. À chaque passage, il effectue une "propagation avant", calcule l'erreur par rapport au résultat attendu, puis ajuste les poids et biais via la rétro propagation pour minimiser cette erreur. La fonction d'activation sigmoïde, appliquée aux valeurs intermédiaires, maintient les résultats entre 0 et 1, et un seuil de 0,5 est utilisé pour interpréter le résultat final en tant que 0 ou 1.

- **Vérification des Arguments et Affichage des Résultats :**

Une fois le réseau entraîné, l'utilisateur peut saisir les valeurs de A et B en ligne de commande pour obtenir une prédiction. Le programme s'assure que les entrées sont bien 0 ou 1 ; autrement, il indique une "valeur incorrecte". Si les entrées sont valides, le réseau calcule la prédiction en appliquant la fonction d'activation sur la sortie finale et affiche le résultat.

5.2.8 Les tâches restantes

- **Prétraitement complet (avec rotation automatique) :**

Cette étape consiste à préparer l'image en ajustant l'orientation de la grille de manière automatique pour un traitement optimal.

- **Réseau de neurones complet et fonctionnel :**

- **Apprentissage :**

Le réseau sera entraîné sur un ensemble de données de lettres afin de reconnaître les caractères dans la grille.

- **Reconnaissance des lettres de la grille et de la liste de mots :**

Le réseau utilisera ses connaissances pour identifier chaque lettre dans la grille et dans la liste

de mots.

- **Reconstruction de la grille :**
Après la reconnaissance, chaque lettre détectée sera remplacée dans une structure qui reconstitue la grille d'origine.
- **Reconstruction de la liste de mots :**
Cette étape consiste à organiser les mots reconnus en une liste lisible, facilitant ainsi la recherche.
- **Résolution de la grille :**
Le programme utilise un algorithme de recherche pour localiser chaque mot de la liste dans la grille reconstruite.
- **Affichage de la grille :**
La grille avec les mots trouvés sera affichée pour visualiser les résultats.
- **Sauvegarde du résultat :**
Une fois la résolution terminée, les résultats seront sauvegardés dans un fichier pour consultation ultérieure.
- **Interface graphique permettant d'utiliser tous ces éléments :**
Une interface utilisateur intuitive permettra d'interagir avec toutes les étapes du programme, de l'importation de l'image à la sauvegarde des résultats.

6 États d'avancement du projet

6.1 Le chargement d'une image et suppression des couleurs

Nous sommes heureux d'annoncer que la phase de chargement d'image a été finalisée avec succès. Cette étape a permis d'intégrer la fonctionnalité de chargement d'images dans notre projet, garantissant ainsi une base solide pour les étapes ultérieures.

Cependant, bien que le chargement des images fonctionne comme prévu, nous avons identifié que le processus de suppression des couleurs nécessite encore des améliorations, notamment aux niveaux les plus fins. Actuellement, la méthode de suppression des couleurs appliquée ne parvient pas à gérer efficacement certaines nuances, ce qui peut affecter la qualité des résultats pour des images complexes.

Nous allons donc concentrer nos efforts sur l'optimisation de cette fonctionnalité afin d'assurer une suppression des couleurs plus précise et cohérente. Cela nous permettra d'améliorer la robustesse de notre système de traitement d'image et de préparer le terrain pour les étapes suivantes du projet.

6.2 La rotation manuelle de l'image

Cette tâche est complétée, mais une amélioration de la qualité de l'image après rotation est envisagée pour une meilleure précision. Par ailleurs, une rotation automatique basée sur le réseau de neurones sera également à développer pour faciliter le prétraitement.

6.3 La détection de la position

La détection de la position de la grille dans l'image a été un défi considérable pour notre équipe. Malheureusement, nous n'avons pas encore réussi à développer un système efficace pour cette tâche. Nous sommes déçus de ne pas avoir pu finaliser cette étape cruciale, qui est essentielle pour la suite de notre

projet. Cependant, nous tenons à souligner notre engagement à travailler d'arrache-pied afin d'améliorer notre méthode. Pour la prochaine soutenance, nous prévoyons d'explorer différentes approches et techniques de traitement d'image, dans le but de mettre en place un système de détection fiable et robuste.

6.4 Le découpage de l'image

Le découpage de l'image représente également une étape que nous n'avons pas encore complètement maîtrisée. Malgré nos efforts pour concevoir une méthode de découpage efficace, nous avons constaté qu'il reste encore beaucoup à prouver et à ajuster. Cette situation est source de déception pour nous, car nous savons que cette fonctionnalité est indispensable pour le bon fonctionnement de notre projet. Néanmoins, nous sommes déterminés à surmonter ces défis. Nous allons approfondir nos recherches et tester diverses techniques afin d'implémenter un système de découpage performant d'ici la prochaine soutenance. Nous croyons fermement qu'avec un travail assidu, nous pourrions atteindre nos objectifs.

6.5 La preuve de concept d'un réseau de neurones

Cette tâche est finalisée et répond aux objectifs fixés pour la première soutenance. L'étape suivante consistera à intégrer le réseau avec la grille de mots cachés et à préparer des jeux d'images pour l'entraînement et la reconnaissance des lettres dans la grille.

6.6 Tableau d'avancement des tâches

Tâches	Première soutenance	Dernière soutenance
Chargement d'une image	100%	100%
Suppression des couleurs	60%	90%
Rotation de l'image	50%	100%
Prétraitement	Non abordé	75%
Détection de la position grille	X%	90%
Détection de la position de la liste de mots	X%	90%
Détection des cases de la grille	X%	90%
Récupération sous la forme d'image	X%	90%
Réseau de neurones complet et fonctionnel	50%	90%
Résolution de la grille	100%	100%
Affichage de la grille résolue	Non abordé	100%
Sauvegarde de la grille résolue	Non abordé	100%
Interface graphique	Non abordé	90%

TABLE 2 – Tableau d'avancement des tâches

7 Conclusion

Ce projet de reconnaissance de caractères (OCR) a permis de mettre en pratique des compétences en traitement d'images, développement de modules en C, et en conception de réseaux de neurones sim-

plifiés. Nous avons abordé des étapes critiques, telles que le chargement d'images, la suppression des couleurs, la détection de grilles et de lettres, ainsi que l'implémentation d'un algorithme de résolution de mots cachés. La réalisation de ces fonctionnalités a exigé une planification rigoureuse et une répartition des tâches méthodique, ce qui a permis d'atteindre un résultat fonctionnel et adaptable à des contextes similaires.

Le développement d'une preuve de concept d'un mini réseau de neurones capable de réaliser une fonction logique simple ouvre des perspectives pour intégrer des algorithmes plus complexes dans de futurs projets de reconnaissance de caractères. Cette expérience enrichissante nous a non seulement permis de renforcer nos compétences en C, mais elle nous a aussi donné un aperçu des défis liés au traitement d'images dans des environnements contraints.

À l'avenir, l'ajout de fonctionnalités plus avancées de détection de formes et de reconnaissance pourrait être envisagé, en exploitant des réseaux neuronaux de plus grande échelle pour améliorer la précision des résultats.

Petit conseil en OCR : si vous commencez à voir des caractères bizarres, prenez une pause ... ou vérifiez votre prétraitement !

A Annexes

Voici l'arborescence du projet OCR, détaillant les fichiers et dossiers pour les différents modules :

```
→ OCR tree
├── Makefile
├── Makefile_neural
├── Makefile_solver
├── bin
│   └── image_loader
├── data
│   └── examples
│       └── level1.png
├── grid
├── include
│   ├── load_image.h
│   ├── neural_net.h
│   ├── preprocessor.h
│   └── solver.h
├── neural_test
├── obj
│   ├── image_processing.o
│   ├── load_image.o
│   ├── main.o
│   ├── main_neural_net.o
│   ├── main_solver.o
│   ├── neural_net.o
│   └── solver.o
├── solver
└── src
    ├── image_processing.c
    ├── image_processing.o
    ├── load_image.c
    ├── load_image.o
    ├── main.c
    ├── main.o
    ├── main_neural_net.c
    ├── main_neural_net.o
    ├── main_solver.c
    ├── main_solver.o
    ├── neural_net.c
    └── neural_net.o
```

FIGURE 2 – Arborescence du projet OCR


```
→ OCR ./solver grid HORIZONtal
(0,0)(9,0)
→ OCR ./solver grid EPITA
Not Found
→ OCR ./solver grid OCR
Not Found
→ OCR cat grid
HORIZONTAL
DXRAHCLBGA
DIKCILEOKC
IGAJHYLYHI
HGFGODTIOT
GDLROWKBFR
PLNRDNERGE
JHAIDUAJGV
UKGFFOLLEH
```

FIGURE 3 – Preuve de la résolution de la grille à partir d'un fichier